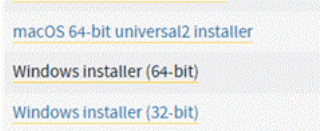
Building Python Application

In this practice we will install Python, VSCode and develop the application

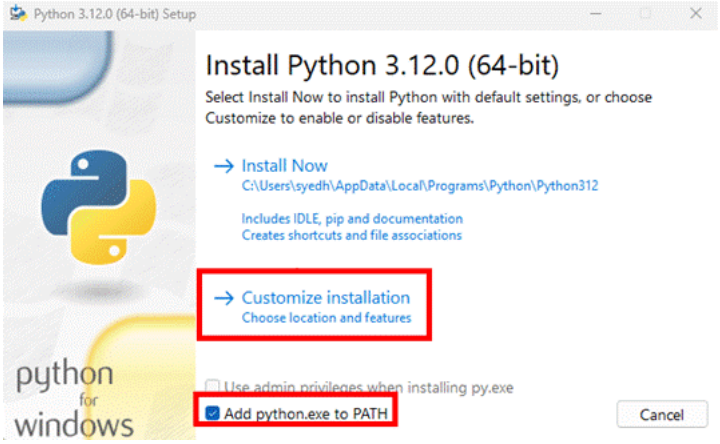
Practice A - Installing Python and Configuring VSCode

1. Install Python
    1. Browse the link https://www.python.org/downloads/release/python-3120/ using browser
    2. Scroll down, locate Windows 64-bit Installer → Click on 'Windows Installer (64-bit)' to download it

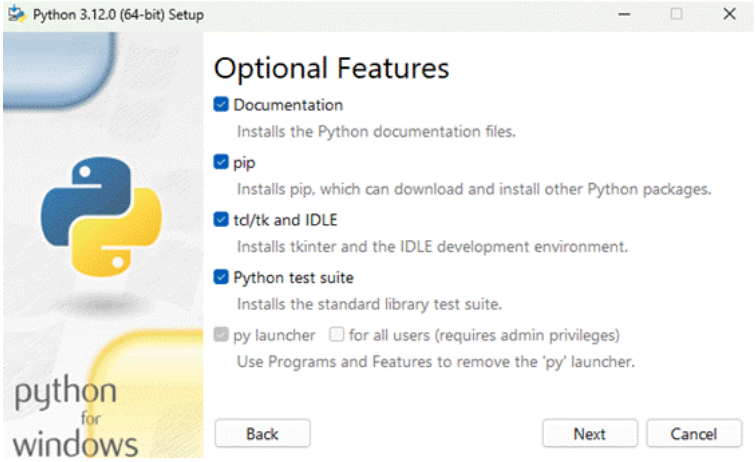
    

    3. Double click on the downloaded executable, to launch the setup

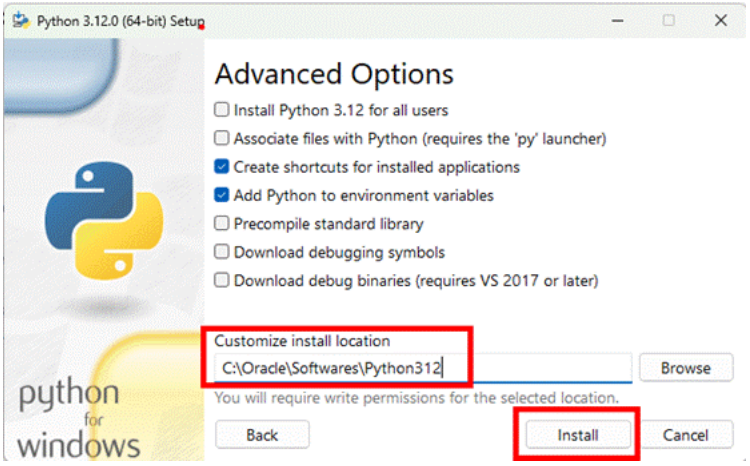
    4. Check 'Add python.exe to Path' box → Click on 'Custom Installation'

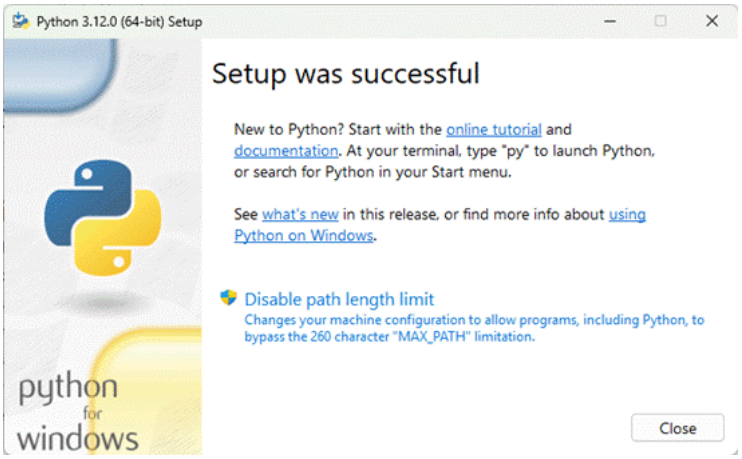
    5. Don't change anything here → Click Next

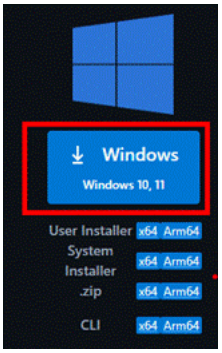
6. Change the installation path to C:\Oracle\Softwares\Python13 → Click Install



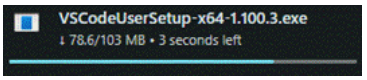
7. Click Close button, to close the installation



2. Install VSCode & install Python extension
   1. Browse the link https://code.visualstudio.com/download
   2. Click on 'Windows' button to begin download

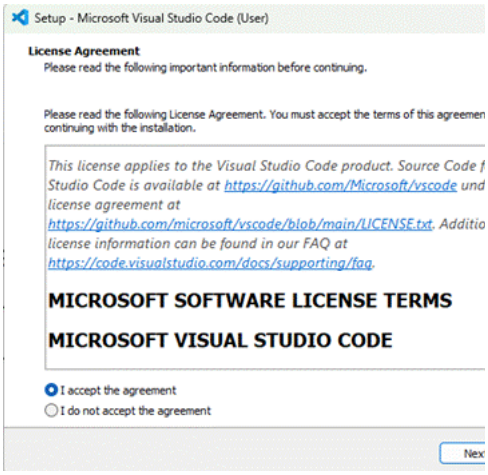


   3. Wait until the download is completed

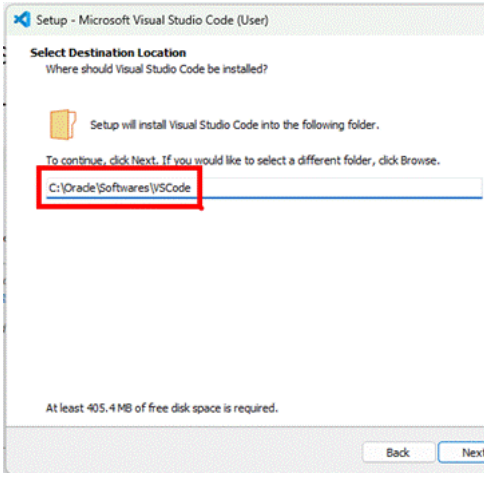


   4. Once downloaded, double click on VSCodeUserSetup-x64 executable file
   5. Answer the wizard as below

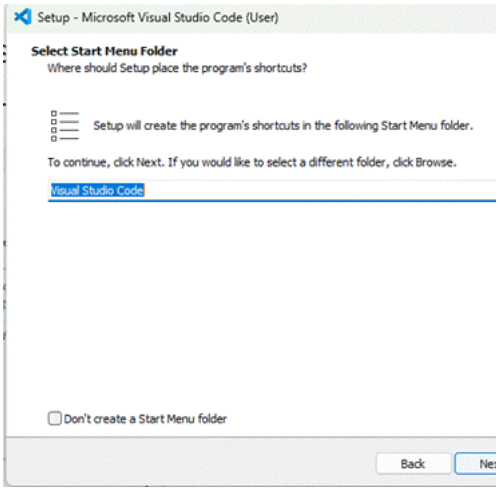
Accept the agreement



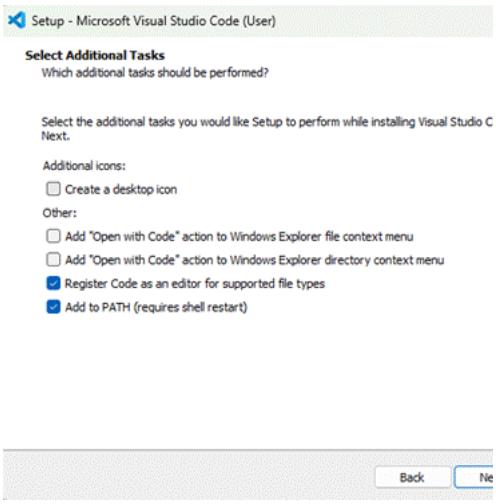
Provide installation Path to C:\Oracle\Softwares



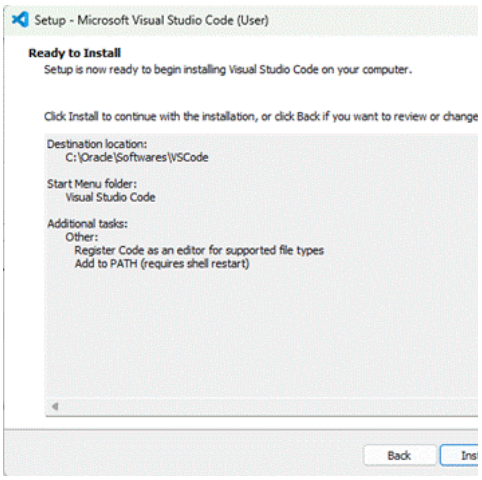
Click Next



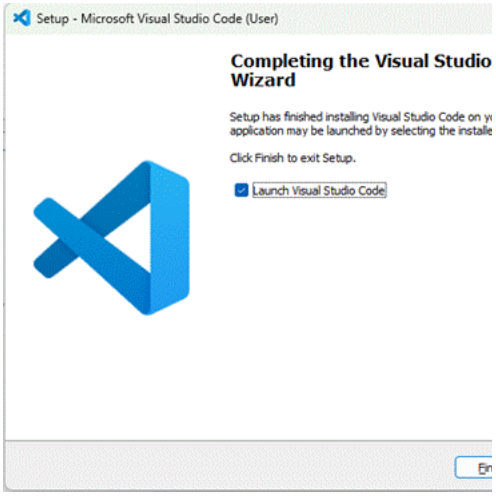
Click Next



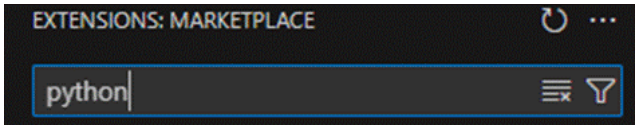
Click Install



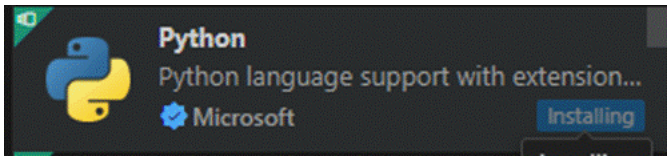
Keeping the 'Launch VSCode' box checked → Click Finish



1. Launch VSCode
2. Within VSCode, Click View → Click Extensions
3. Locate the 'search' box over left top corner, Search for python

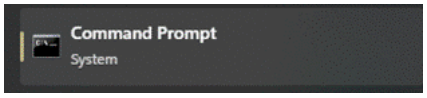


4. Click on 'Install' button



5. Close the VSCode

3. Create Labs Folder
    1. Launch a command prompt (Search for command prompt over windows start menu → type command prompt → click on command prompt)

    2. Create labs folder using below commands

    ```
    C:
    mkdir -r c:\labs\ai-devops
    cd c:\labs\ai-devops
    ```

4. Create and Activate a Virtual Environment

    a. Verify if python is installed using below command

    ```
    python --version
    ```

    1. Using the command prompt you have just launched execute below command

    ```
    cd c:\labs\ai-devops
    python -m venv .venv
    ```

    Note: Here we are creating an environment called '.venv'. Above command should have created a folder by name '.venv' within 'c:\labs\ai-devops'

    2. Activate the .venv environment using below command

    ```
    .\.venv\Scripts\activate
    ```

    Note: notice the prompt has changed, this indicates we are working on '.venv' environment.

    ```
    (.venv) c:\labs\ai-devops>
    ```

    3. Deactivate the '.venv' environment, by executing below command

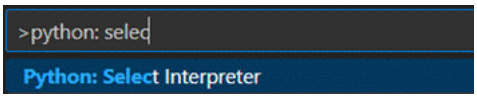
    ```
    deactivate
    ```

    Notice, that prompt has changed again

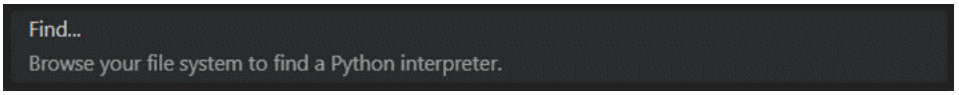
    ```
    (.venv) c:\labs\ai-devops>deactivate
    c:\labs\ai-devops>
    ```

    4. Close the command prompt.

5. Code and run your first python program
    1. Launch the VSCode
    2. Configure Python of '.venv' environment as Interpreter
        1. Within VSCode, press Ctrl+Shift+p
        2. Look for 'Python: Select Interpreter' → Click on the list item 'Python: Select Interpreter'

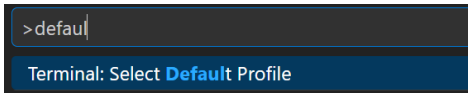
        3. Click on 'Enter Interpreter Path'
        4. Click on 'Find' list item

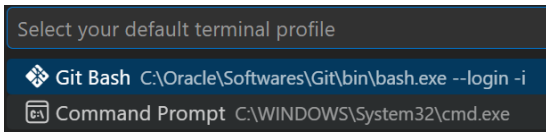
        5. Browse the path 'c:\labs\ai-devops\.venv\Scripts' → Select 'python.exe' → Click 'Select Interpreter'. For Mac machines 'Scripts' folder is replaced by 'bin' folder.
        6. We have configured python.exe from '.venv' environment as our interpreter for execution.
    3. Create a new file named main.py
        1. Right click within 'ai-devops' folder, but outside .venv → Click New File
        2. Enter the name as 'main.py' → Press Enter key
        3. Enter the code within main.py as below

        4. Save the changes using 'Ctrl+S' OR File → Save
    4. This is applicable for 'Windows' machines. By default, in VSCode Powershell is used as the default command interpreter. Change it to command prompt using below steps.
        a. Within VSCode, press Ctrl+Shift+p

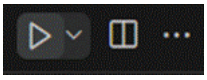
b. Look for ‘Terminal: Default Profile’ → Click on it



c. Click on ‘Command Prompt’



5. Execute the main.py
   1. Over the right top corner, locate the Play icon → Click on it



   2. Notice, our python program is executed and output is printed

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\labs\ai-devops>c:\labs\ai-devops\.venv\Scripts\activate.bat

(.venv) C:\labs\ai-devops>c:\labs\ai-devops\.venv\Scripts\python.exe c:/labs/ai-devops/main.py
Congratulations! Welcome to Python Programming
```

6. Close main.py file

Practice B - Understanding Python Application with vulnerabilities

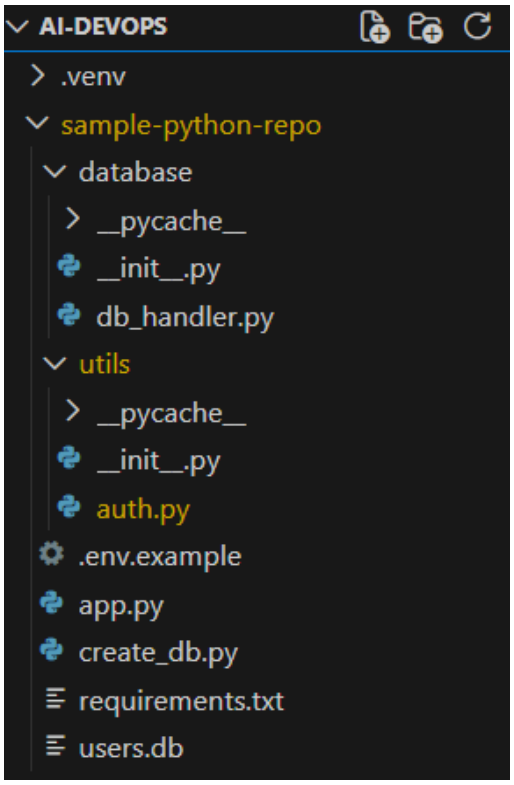
This workshop aims to help developers understand the structure and flow of a Python Flask application by examining the code organization, route definitions, function implementations, and inter-file dependencies.

Steps Summary:

1. Download the Application from instructor provided link
2. Examine Application Entry Point and Route Definitions
3. Review Authentication Utility Functions
4. Explore Database Handler Functions
5. Execute the Flask Application
6. Test with Two Use Cases
7. Describe the Output

Step 1: Download the Application from instructor provided link

1. Browse the link provided by instructor for material
2. Download 'sample-python-repo' zip file
3. Extract 'sample-python-repo.zip' to C:\labs\ai-devops folder
4. The folder should be visible in VSCode as below



Step 1: Examine Application Entry Point and Route Definitions

Problem Statement

The main application file serves as the entry point for the Flask server. We need to understand how routes are defined, how imports are organized, and how the application is configured to run.

Steps

a. Open the Python file named [app.py]
b. Review the file by following the steps below

Step 1.1: Imports and Application Initialization

```
import os
import sqlite3
```

```
from flask import Flask, request, jsonify
from utils.auth import verify_token
from database.db_handler import execute_query

app = Flask(__name__)

SECRET_KEY = "hardcoded-super-secret-key-12345"

app.config['DEBUG'] = True
```

This section imports necessary modules and initializes the Flask application. The verify_token function is imported from the utils.auth module, and execute_query is imported from the database.db_handler module. The SECRET_KEY variable stores a string value used for authentication, and the DEBUG configuration is set.

Code	Explanation
`from flask import Flask, request, jsonify`	Imports Flask classes and functions for creating the app, handling requests, and returning JSON responses
`from utils.auth import verify_token`	Imports token verification function from the auth utility module
`from database.db_handler import execute_query`	Imports database query execution function from the db_handler module
`app = Flask(__name__)`	Creates an instance of the Flask application class
`SECRET_KEY = "hardcoded-super-secret-key-12345"`	Assigns a string value to the SECRET_KEY variable
`app.config['DEBUG'] = True`	Sets the debug mode configuration property to True

Step 1.2: User Lookup Route

```
@app.route('/user/<username>')
def get_user(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = f"SELECT * FROM users WHERE username = '{username}'"
    cursor.execute(query)
    user = cursor.fetchone()
    conn.close()
    return jsonify(user)
```

This route accepts a username parameter from the URL path and returns user information. The function connects to a SQLite database, creates a cursor for executing queries, constructs a SELECT statement using the provided username, executes it, retrieves one result, closes the connection, and returns the result as JSON.

Code	Explanation
`@app.route('/user/<username>')`	Decorator that maps the URL pattern to the function below, capturing the username from the URL
`def get_user(username):`	Function definition that receives the captured username as a parameter
`conn = sqlite3.connect('users.db')`	Establishes a connection to the SQLite database file named users.db
`cursor = conn.cursor()`	Creates a cursor object used to execute SQL statements
`query = f"SELECT * FROM users WHERE username = '{username}'"`	Creates a SQL query string using f-string formatting
`cursor.execute(query)`	Sends the SQL query to the database for execution
`user = cursor.fetchone()`	Retrieves the first row from the executed query results
`conn.close()`	Closes the database connection
`return jsonify(user)`	Converts the user data to JSON format and returns it as the HTTP response

Step 1.3: Login Route

```
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'")
    user = cursor.fetchone()
    conn.close()

    if user:
        return jsonify({"status": "success", "token": SECRET_KEY})
    return jsonify({"status": "failed"}), 401
```

This route handles POST requests for user login. It extracts JSON data from the request, retrieves username and password values, queries the database to find a matching user, and returns a success response with a token or a failure response with status code 401.

Code	Explanation
`@app.route('/login', methods=['POST'])`	Defines an endpoint that only accepts HTTP POST requests
`data = request.get_json()`	Parses the JSON body from the incoming HTTP request
`username = data.get('username')`	Extracts the username field from the parsed JSON data
`password = data.get('password')`	Extracts the password field from the parsed JSON data
`cursor.execute(f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'")`	Executes a SQL query with the username and password values embedded in the string
`if user:`	Checks if a matching user record was found
`return jsonify({"status": "success", "token": SECRET_KEY})`	Returns a JSON response with success status and the SECRET_KEY as token
`return jsonify({"status": "failed"}), 401`	Returns a JSON response with failure status and HTTP 401 status code

Step 1.4: Delete User Route with Try-Except Block

```
@app.route('/admin/delete')
def delete_user():
    try:
        user_id = request.args.get('id')
        execute_query(f"DELETE FROM users WHERE id = {user_id}")
    except:
        pass

    return jsonify({"status": "deleted"})
```

This route accepts a user ID as a query parameter and deletes the corresponding user. It uses a try-except block to handle any errors during execution, and always returns a success response regardless of whether the deletion actually occurred.

Code	Explanation
`@app.route('/admin/delete')`	Defines an endpoint for deleting users
`user_id = request.args.get('id')`	Retrieves the 'id' parameter from the URL query string
`execute_query(f"DELETE FROM users WHERE id = {user_id}")`	Calls the imported execute_query function with a DELETE SQL statement
`except:`	Catches any exception that occurs in the try block
`pass`	Does nothing when an exception is caught
`return jsonify({"status": "deleted"})`	Always returns a JSON response indicating deletion was successful

Step 1.5: Data Route with Pickle Deserialization

```
@app.route('/data')
def get_data():
    import pickle
    data = request.args.get('payload')
    return pickle.loads(data.encode())
```

This route accepts a payload parameter from the query string, encodes it, and deserializes it using the pickle module. The result of the deserialization is returned directly as the response.

Code	Explanation
@app.route('/data')	Defines an endpoint for retrieving processed data
import pickle	Imports the pickle module for serialization and deserialization
data = request.args.get('payload')	Retrieves the 'payload' parameter from the URL query string
return pickle.loads(data.encode())	Converts the payload string to bytes and deserializes it using pickle.loads, returning the deserialized object

Step 1.6: Application Execution Block

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

This block checks whether the script is being run directly (not imported as a module) and starts the Flask development server on all network interfaces at port 5000.

Code	Explanation
if __name__ == '__main__':	Condition that evaluates to True only when the file is executed directly
app.run(host='0.0.0.0', port=5000)	Starts the Flask development server, binding to all available network interfaces on port 5000

Step 2: Review Authentication Utility Functions

Problem Statement

The authentication module provides functions for generating and verifying tokens. We need to understand how these functions work and how they interact with the main application.

Steps

a. Open the Python file named auth.py inside the utils folder
b. Review the file by following the steps below

Step 2.1: Imports and Token Generation

```
import base64

def generate_token(user_id):
    token = base64.b64encode(f"{user_id}:{int(time.time())}".encode())
    return token.decode()
```

This function creates a token by combining a user ID with the current timestamp, encoding the result in base64, and returning it as a string.

Code	Explanation
`import base64`	Imports the base64 module for encoding and decoding operations
`def generate_token(user_id):`	Defines a function that takes a user ID as a parameter
`f"{user_id}:{int(time.time())}".encode()`	Creates a string combining user_id and current timestamp, then encodes it to bytes
`base64.b64encode(...)`	Converts the bytes to a base64 encoded format
`return token.decode()`	Converts the base64 bytes to a string and returns it

Step 2.2: Token Verification

```
def verify_token(token):
    try:
        decoded = base64.b64decode(token).decode()
        user_id = decoded.split(':')[0]
        return user_id
    except:
        return None
```

This function attempts to decode a base64 token, extract the user ID portion, and return it. If any error occurs during this process, it returns None.

Code	Explanation
`def verify_token(token):`	Defines a function that takes a token string as input
`base64.b64decode(token).decode()`	Decodes the base64 token and converts it to a string
`user_id = decoded.split(':')[0]`	Splits the decoded string at the colon and takes the first part as user ID
`return user_id`	Returns the extracted user ID
`except:`	Catches any exception that occurs during decoding or splitting
`return None`	Returns None when an exception is caught

Step 2.3: Additional Function with Import

```
def some_function():
    import datetime
    return True
```

This function imports the datetime module inside its body and returns True. The imported module is not used elsewhere in the function.

Code	Explanation
`def some_function():`	Defines a function named some_function
`import datetime`	Imports the datetime module within the function scope
`return True`	Returns the boolean value True

Step 3: Explore Database Handler Functions

Problem Statement

The database handler module contains functions for executing SQL queries. We need to understand how queries are passed and executed, and how results are returned.

Steps

a. Open the Python file named db_handler.py inside the database folder
b. Review the file by following the steps below

Step 3.1: Query Execution Function

```
import sqlite3

def execute_query(query, params=None):
    conn = sqlite3.connect('app.db')
    cursor = conn.cursor()

    if params:
        cursor.execute(query, params)
    else:
        cursor.execute(query)

    result = cursor.fetchall()
    conn.commit()
    conn.close()
    return result
```

This function establishes a connection to the database, executes a query with optional parameters, retrieves all results, commits the transaction, closes the connection, and returns the results.

Code	Explanation
`import sqlite3`	Imports the SQLite3 database module
`def execute_query(query, params=None):`	Defines a function that accepts a query string and optional parameters
`conn = sqlite3.connect('app.db')`	Creates a connection to the app.db SQLite database
`cursor = conn.cursor()`	Creates a cursor object for executing SQL statements
`if params:`	Checks whether parameters were provided
`cursor.execute(query, params)`	Executes the query with the provided parameters
`cursor.execute(query)`	Executes the query without parameters
`result = cursor.fetchall()`	Retrieves all rows from the executed query
`conn.commit()`	Commits the current transaction to save changes
`conn.close()`	Closes the database connection
`return result`	Returns the fetched results

Step 3.2: User Retrieval Function

```
def get_user_by_id(user_id):
    try:
        result = execute_query(f"SELECT * FROM users WHERE id = {user_id}")
        if result:
            return result[0]
        return None
    except Exception as e:
        return str(e)
```

This function attempts to retrieve a user by their ID using the execute_query function. If a result is found, it returns the first row; otherwise, it returns None. If an exception occurs, it returns the exception as a string.

Code	Explanation
`def get_user_by_id(user_id):`	Defines a function that accepts a user ID parameter
`try:`	Begins a try block for exception handling
`result = execute_query(f"SELECT * FROM users WHERE id = {user_id}")`	Calls execute_query with a SELECT statement containing the user ID
`if result:`	Checks if the result contains any data
`return result[0]`	Returns the first row from the result set
`return None`	Returns None if no results were found
`except Exception as e:`	Catches any exception and stores it in variable e
`return str(e)`	Returns the string representation of the exception

Step 4: Execute the Flask Application

To run the application, install the required dependencies and execute the main file.

Dependency Installation

a. Please disconnect from VPN
b. Launch a command prompt
c. Execute below commands

```
cd c:\labs\ai-devops
.venv\Scripts\activate
(.venv) c:\labs\ai-devops> cd sample-python-repo
(.venv) c:\labs\ai-devops\sample-python-repo>pip install -r requirements.txt
```

Running the Application

```
(.venv) c:\labs\ai-devops\sample-python-repo>python app.py
```

The application will start and display output similar to:

```
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.0.39:5000
```

Step 5: Testing with Two Use Cases

Test Case 1: Retrieve a User

```
curl http://127.0.0.1:5000/user/admin
```

This request sends a GET request to the user endpoint with the username "john". The application will process the request and return the user data if found.

Test Case 2: Login Request

On windows machine command prompt use:

```
curl -X POST http://localhost:5000/login -H "Content-Type: application/json" -d
"{\"username\": \"admin\", \"password\": \"admin123\"}"
```

On Mac machine terminal use:

```
curl -X POST http://127.0.0.1:5000/login -H "Content-Type: application/json" -d
'{"username": "admin", "password": "admin123"}'
```

This request sends a POST request to the login endpoint with a JSON body containing username and password. The application will query the database and return a response based on whether a matching user is found.

Step 6: Describe the Output

Output for Test Case 1 (User Retrieval)

If a user named "admin" exists in the users.db database, the response will be:

```
C:\Users\syedh>curl http://127.0.0.1:5000/user/admin
[
  1,
  "admin",
  "admin123"
]
```

If the user does not exist, the response will be:

```
C:\Users\syedh>curl http://127.0.0.1:5000/user/john
null
```

Output for Test Case 2 (Login)

If the credentials match an existing user, the response will be:

```
{
  "status": "success",
  "token": "hardcoded-super-secret-key-12345"
}
```

If the credentials do not match any user, the response will be:

```
{
  "status": "failed"
}
```

with an HTTP status code of 401.

Key Takeaways:

- Flask applications use decorators like @app.route to map URLs to Python functions
- Request data can be accessed through request.get_json() for JSON bodies or request.args.get() for query parameters
- Functions can be organized across multiple files and imported using relative imports
- Database connections require explicit management including connection, cursor creation, commit, and close operations
- Exception handling using try-except blocks allows functions to control error flow
- The if __name__ == '__main__' block controls whether the application runs when the file is executed directly